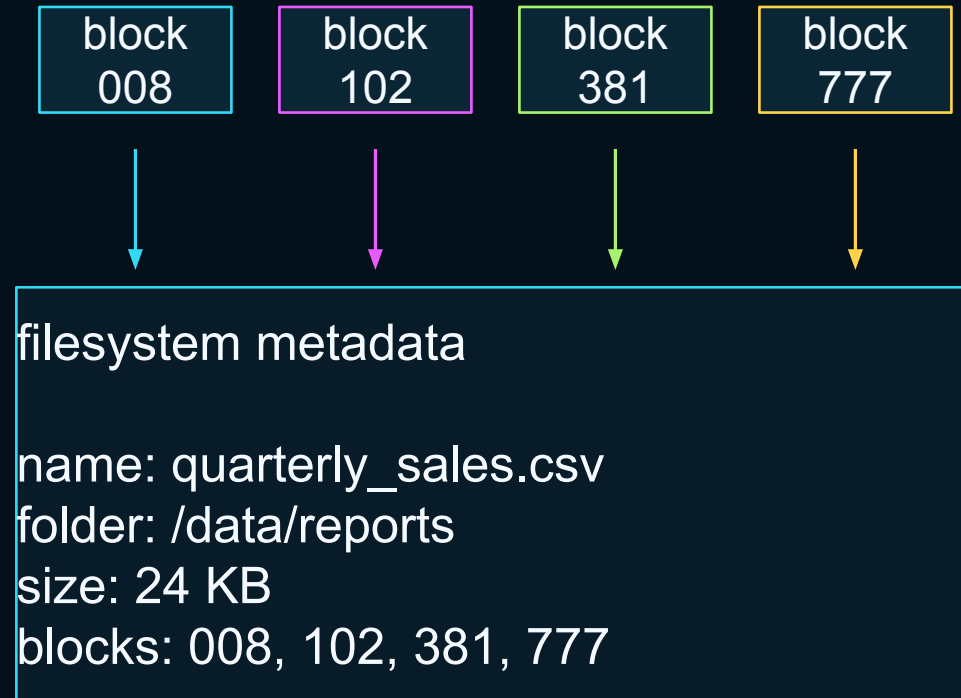


WHERE DID MY FILE GO?

Memory, Storage, and Filesystems



A drive stores blocks. The filesystem supplies names, folders, permissions, and meaning.

30 min instruction • 30 min paper filesystem exercise

Cold open: what changed?

The user sees a simple rename. The machine updates a record.

report1.csv



quarterly_sales.csv

Did every byte of the file
get rewritten?

User view
“I changed the name.”

Filesystem view
“I updated metadata.”

Storage view
“The blocks are still
blocks.”

renaming usually changes metadata, not the file contents.

Today's mission

Build a mental model for memory while programs run and storage after they stop.

- 1 Separate** temporary memory from permanent storage
- 2 Name** RAM, process, stack, heap, block, sector
- 3 Explain** how files are tracked with metadata
- 4 Simulate** a tiny paper filesystem

KEY IDEA

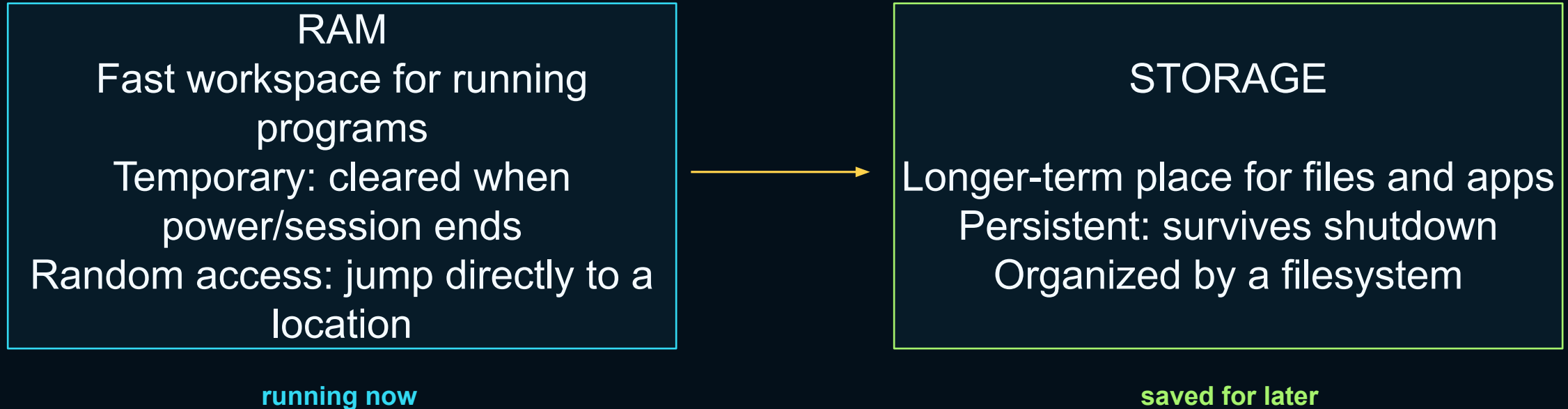
A file's name is not the same thing as its contents.

Names, folders, and permissions are metadata that point to data.

This block connects user-friendly file behavior to what actually happens underneath.

Temporary memory vs. permanent storage

Two places for data, two very different jobs.



When you open a file, data often moves from storage into memory so a program can work with it.

Variables: names associated with values in memory

A variable is not a little box with ink on it; it is a name the runtime can resolve to a value.

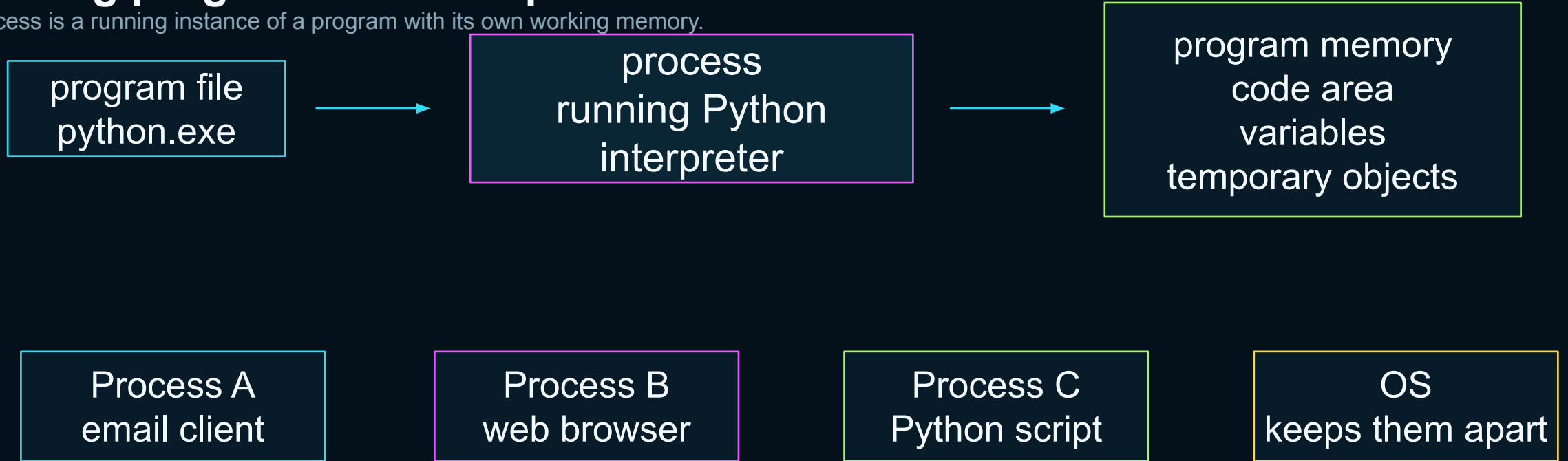
```
price = 12  
quantity = 3  
total = price *  
quantity
```



The exact memory layout is managed for you — especially in Python.

Running programs become processes

A process is a running instance of a program with its own working memory.



The operating system manages many processes and gives each the illusion of its own memory space.

Stack and heap: a friendly first pass

Do not memorize internals yet — just know that program memory has regions with different jobs.

STACK

Think: short-lived work list

Function calls

Local bookkeeping

Usually grows and shrinks predictably

HEAP

Think: flexible object area

Data created while program runs

Managed by the runtime

Can be more dynamic

```
def add_tax(price):  
    return price * 1.07
```

Python hides most of this from beginners, but the idea helps explain “memory used by a program.”

Storage devices: SSDs and hard drives

Different physical technology, same user-facing promise: store bytes and retrieve them later.

SSD

No spinning platter
Very fast random access
Stores data in flash memory cells
Common in modern laptops

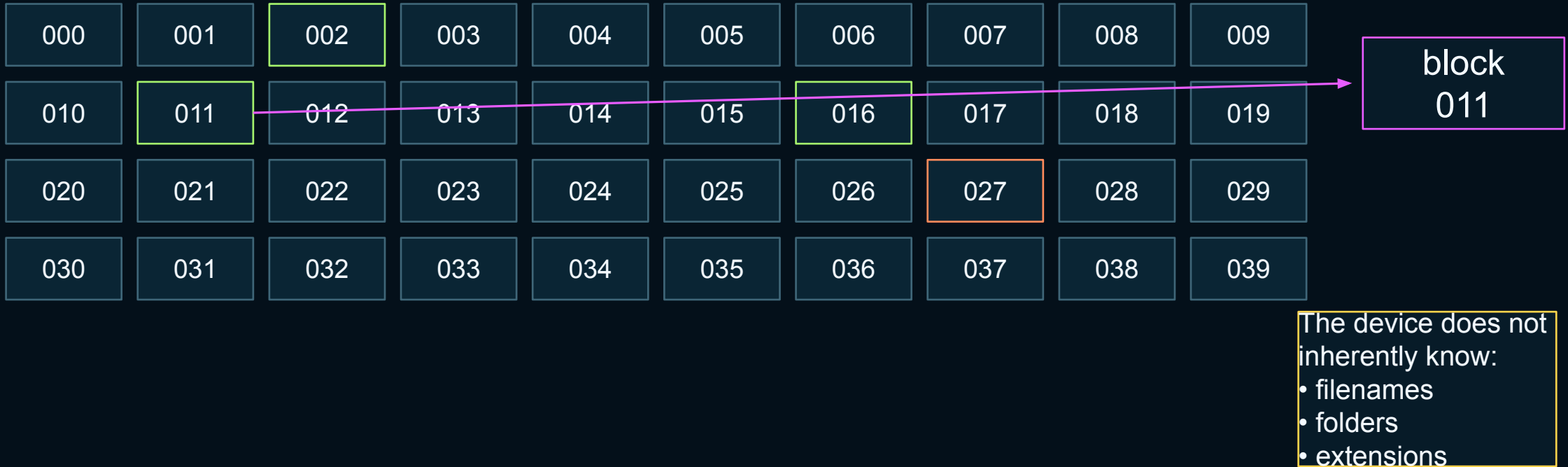
HARD DRIVE

Spinning magnetic platters
Mechanical movement matters
Still common for cheap large capacity
Historically used sectors



A practical mental model: numbered blocks

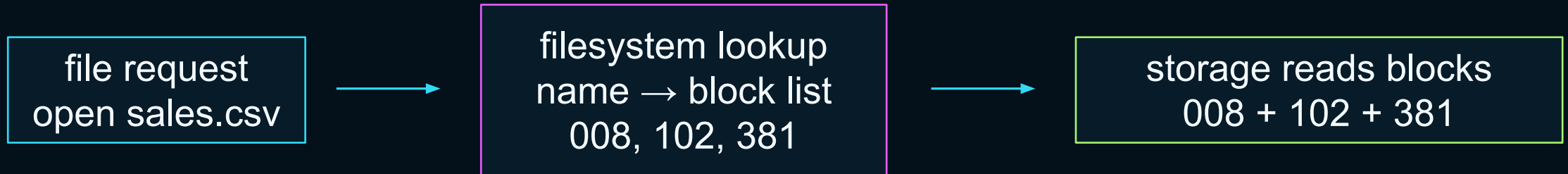
Imagine storage as a giant collection of numbered locations.



Blocks give the operating system places to read and write. The filesystem gives them organization.

Reading and writing blocks of data

Files can be larger than one block, so the system keeps track of all the pieces.



Block 008: header/data

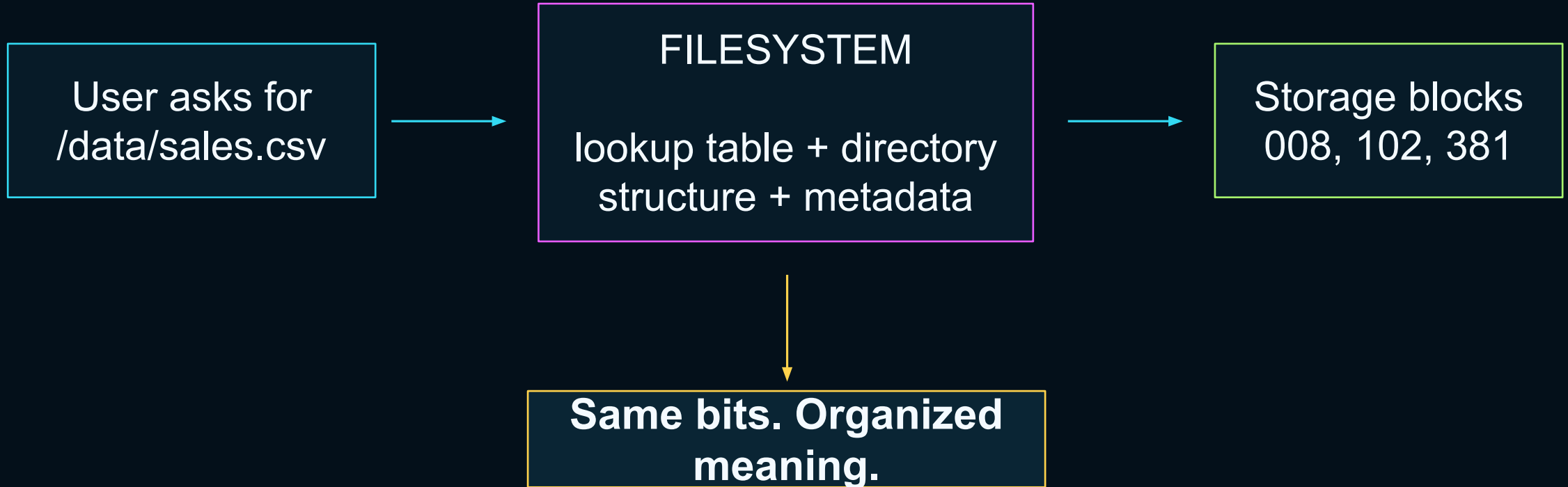
Block 102: more data

Block 381: final data

To the user it looks like one file. Underneath, it may be many blocks.

The filesystem is the map

It maintains records that connect names, folders, permissions, and block locations.



Filesystem metadata: the file's record card

The metadata is data about the file, not necessarily the file contents.

name

quarterly_sales.csv

size

24 KB

block list

008, 102, 381

timestamps

created • modified

permissions

who can read/write

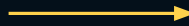
folder

/data/reports

```
metadata: { name, parent_folder, size, timestamps, permissions,  
            blocks }
```

Folders are logical organization

A path is a route through the directory structure.



absolute path
/data/reports/sales.csv

folder relationship
reports contains sales.csv

file record
points to data blocks elsewhere

Folders make storage understandable for humans. The data blocks do not need to sit “inside” a visual folder.

File names and extensions

Extensions are a useful hint, not magic.

`sales.csv`

`photo.jpg`

`notes.txt`

`song.mp3`

`.csv`

interpret as
rows/columns

`.jpg`

interpret as compressed
image

`.txt`

interpret as text

`.mp3`

interpret as
audio

Changing `.jpg` to `.txt` does not magically convert the underlying image into plain text.

SHOW THE RICK ASTLEY IMAGE

Fragmentation: one file, scattered blocks

A file can exist even if its blocks are not next to each other.



The filesystem's block list lets scattered pieces appear as one normal file.

Deleting a file: removing the map entry

Deletion often marks space as available before every old bit is overwritten.



old blocks: 008, 102, 381 status: available for reuse

That is why “deleted” does not always mean “physically erased immediately.”

Why rename and same-drive move can be fast

Often, the system changes metadata while leaving the data blocks alone.

report1.csv →
quarterly_sales.csv

rename
metadata: name changes
blocks: usually unchanged

/downloads/a.csv →
/reports/a.csv

move on same drive
metadata: parent folder changes
blocks: often unchanged

Moving to another drive usually requires copying the bytes to different storage, then deleting the old record.

What happens when a program opens a file by name?

The name starts a lookup. The blocks provide the contents.



```
open("/data/reports/sales.csv") → lookup metadata → read blocks →  
bytes returned to program
```

The program does not scan the entire drive for a printed label named sales.csv.

Guided exercise: paper filesystem

Students become the operating system. The table is the map; the blocks are the storage.

01
Block map

02
File table

03
Folder tree

04
Tasks

Blocks

00	01	02	03	04
05	06	07	08	09
10	11	12	13	14

Rules

- update metadata
- track free blocks
- do not rewrite data unless told

Goal

Explain each operation in plain English before doing it.

Paper filesystem reference card

Use this simplified table for the next tasks.

FILE TABLE

Name	Folder	Size	Blocks
sales.csv	/data	3 KB	02, 11, 16
logo.png	/images	2 KB	04, 05
notes.txt	/	1 KB	08
old.tmp	/data	2 KB	21, 22

FOLDER TREE

```
/
├── data
│   ├── sales.csv
│   └── old.tmp
├── images
│   └── logo.png
└── notes.txt
```

USED BLOCKS: 02 04 05 08 11 16 21 22
13 14 15 ...

FREE BLOCKS: 00 01 03 06 07 09 10 12

Assume these files are all on the same drive.

EXERCISE

Round 1: locate, rename, move

Work only by updating the simplified filesystem records.

A

Locate the blocks for
sales.csv

B

Rename
old.tmp → archive.tmp

C

Move notes.txt
from / to /data

Write the updated file table rows.
Do any data blocks change? Explain why or why not.

Challenge: circle each operation that should be fast on the same drive.

EXERCISE

Round 1 answer: metadata changed, blocks stayed

The file contents did not need to move.

- A) sales.csv blocks: 02, 11, 16
- B) old.tmp becomes archive.tmp
Folder stays /data; size stays 2 KB; blocks stay 21, 22
- C) notes.txt moves to /data
Folder changes from / to /data; size stays 1 KB; block stays 08

Rename and same-drive move are usually metadata operations, so they can be very fast.

EXERCISE

Round 2: delete and open by name

Now reason about what becomes available and how lookup works.

A

Delete logo.png.
Which blocks become
available?

B

Explain why deleting may
not immediately erase
every bit.

C

Program opens
/data/sales.csv.
What steps happen?

Use the file table + folder tree + block list.
Answer in plain English before changing the table.

Extra: what would be different if sales.csv were moved to a USB drive?

Round 2 answer + discussion

Close the loop: file names, contents, and metadata are different things.

A) Deleting `logo.png` removes its file entry.

Blocks 04 and 05 become available.

B) The bits in 04 and 05 may remain until reused.

The filesystem just no longer treats them as `logo.png`.

```
C) open("/data/sales.csv")
1. find /
2. find data folder
3. find sales.csv metadata
4. read blocks 02, 11, 16
5. return bytes to the program
```

Why does a computer need a filesystem?

What is the difference between a file name and file contents?

Why can same-drive moves be fast, while cross-drive moves can take longer?

How can one file be stored in non-continuous blocks?